Научно исследовательский университет ИТМО
Факультет систем управления и робототехники

Отчет по лабораторной работе №3
Матрицы в 3D-графике

Выполнил студент группы R3280 Мовчан И. Е.
Преподаватель Перегудин А. А.

Санкт-Петербург
2024

1 Создание куба

С помощью кода из задания опишем куб в пространстве:

```
verticesCube = [
    -1,  1,  1, -1, -1,  1, 1, -1;
    -1, -1,  1,  1, -1, -1, 1,  1;
    -1, -1, -1, -1,  1,  1, 1,  1;
    1,  1,  1,  1,  1,  1, 1,  1
    ];

facesCube = [
    1, 2, 6, 5;
    2, 3, 7, 6;
    3, 4, 8, 7;
    4, 1, 5, 8;
    1, 2, 3, 4;
    5, 6, 7, 8
];

DrawShape(verticesCube, facesCube, 'blue');
axis equal;
view(3);

function DrawShape(vertices, faces, color)
    patch('Vertices', (vertices(1:3,:)./vertices(4,:))', 'Faces',
   faces, 'FaceColor', color);
end
```

Использование четырехкомпонентного (четвертая координата говорит о «весе») вектора мотивируется удобностью реализации смещения 3D объекта в 4D пространстве. Другие фигуры задаются через определение точек вершин (в кубическом случае за это отвечает переменная verticesCube, в которой мы задаём вершины единичного куба) и их соединение с последующим достраиванием грани через определение точек, находящихся на ней (у нас за это отвечает переменная facesCube).

Функция DrawShape, на вход которой передаются массив вершин, грани и цвет, используется как обёртка для другой функции - path, отрисовывающей трёхмерные фигуры через всё те же параметры.

Строчки axis equal и view(3) указывают на то, что у нас будет выделено равно количество пространства для каждый из осей, а также на то, что мы работаем в 3D и хотим, соответственно в нём и увидеть

наш результат.

Запустив код, получим следующее:

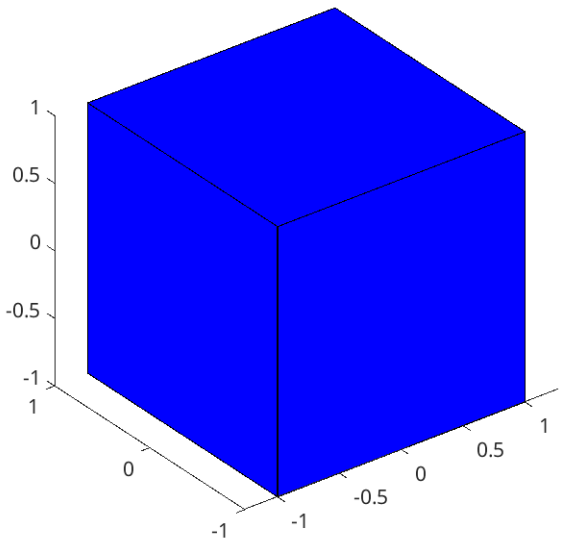


Рис. 1: Результат работы исходного кода

Попробуем теперь немного поэксперементировать с данными. Допустим, мы захотели изменить исходные вершины единичного куба на что-то следующее:

```
verticesCube = [
    -1,  2,  1, -1, -1,  1, 1, -1;
    -1, -1,  2,  1, -1, -1, 1,  1;
    -1, -1, -1, -2,  1,  1, 1,  1;
    1,  1,  1,  1,  1,  1, 1,  1
    ];
DrawShape(verticesCube, facesCube, 'blue');
```

На выходе должны будем получить что-то невнятное (так как вершины изменялись произвольным образом и не изменялись). Результат работы представлен на 2 рисунке.

Таким образом, мы научились строить различные фигуры.

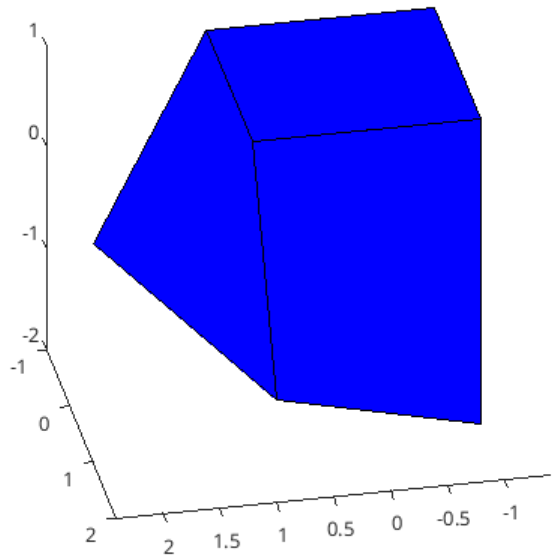


Рис. 2: Не куб

2 Изменение масштабов

Попробуем теперь немного изменить масштабы нашего куба. Матрица данного преобразования имеет следующий вид:

```
Scale = [Sx,  0,  0,  0;
          0, Sy,  0,  0;
          0,  0, Sz,  0;
          0,  0,  0,  1];
```

Здесь переменные Sx, Sy и Sz отвечают за растяжения/сужения по соответствующим осям. В случае простого растяжения по всем осям они будут равны. Последняя переменная не трогается, она сейчас нам не нужна.

Итак, с помощью изменим код и выведем расширенный в 2 раза куб:

```
A1 = [2, 0, 0, 0;
      0, 2, 0, 0;
      0, 0, 2, 0;
      0, 0, 0, 1];
verticesCube = A1*verticesCube;
```

Результат работы будет следующим:

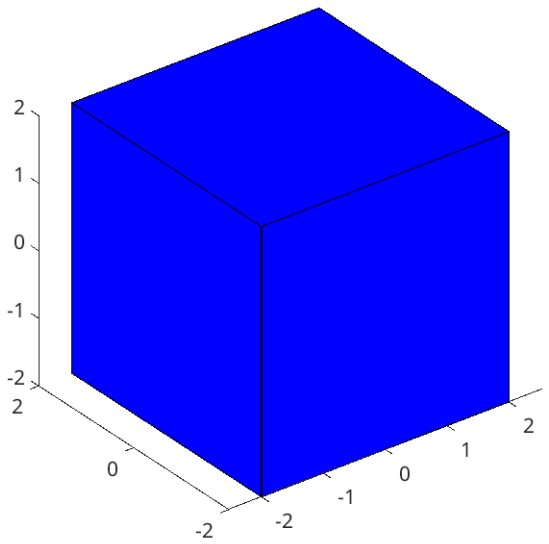


Рис. 3: Масштабированный куб

Получили то, что и хотели! Изменим теперь масштабы только по оси Ox:

```
A1 = [2, 0, 0, 0;
      0, 1, 0, 0;
      0, 0, 1, 0;
      0, 0, 0, 1];
verticesCube = A1*verticesCube;
```

На рисунке 4 приведен результат работы. Всё получилось!

3 Перемещение

Следующая матрица совершает преобразование перемещения:

```
A2 = [1, 0, 0, Tx;
      0, 1, 0, Ty;
      0, 0, 1, Tz;
      0, 0, 0,  1];
verticesCube = A2*verticesCube;
```

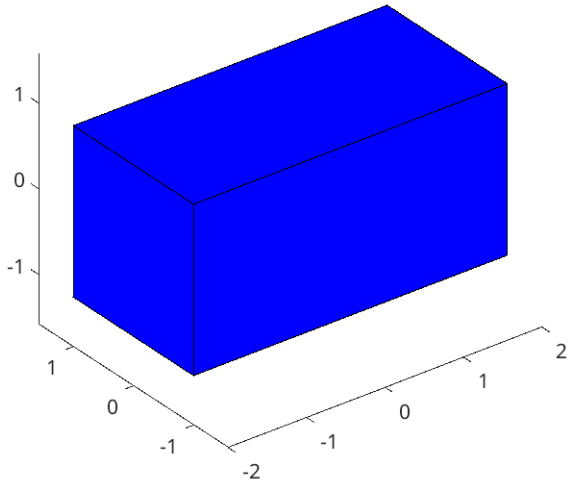


Рис. 4: Масштабированный по Ox куб

Переменные Tx, Ty и Tz отвечают за сдвиг по Ox, Oy и Oz соответственно. Здесь нам как раз требуется четвертая компонента вектора, ведь при умножении вектора на матрицу мы получаем прибавку по соответствующей оси.

Посмотрим на неё в действии:

```
A2 = [1, 0, 0, 1;
      0, 1, 0, 1;
      0, 0, 1, 0;
      0, 0, 0, 1];
verticesCube2 = A2*verticesCube;
DrawShape(verticesCube, facesCube, 'blue');
DrawShape(verticesCube2, facesCube, 'green');
```

Результат представлен на рисунке 5. Получили сдвиг, который и задавали!

Посмотрим теперь на комбинацию преобразований перемещения и масштабирования. Зададим матрицу $A1$, растягивающую наш куб по оси Ox и Oz в 2 раза, и матрицу $A2$, делающей сдвиг центра в $(1, 1, 1)$. Композиция перемещений есть умножение матриц, а поэтому:

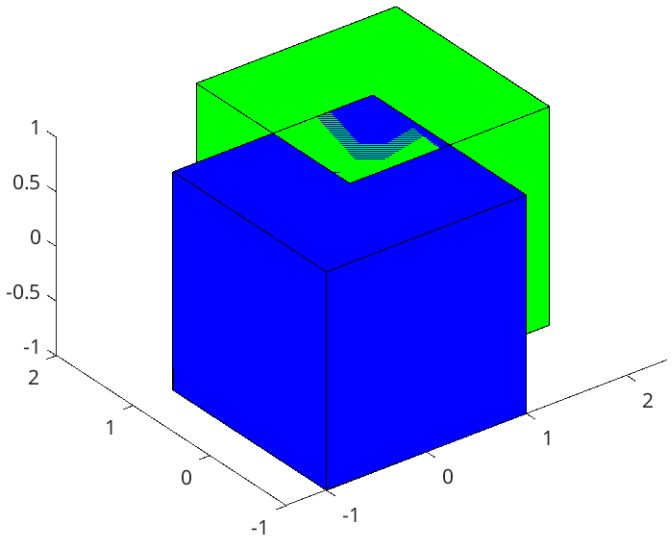


Рис. 5: Сдвиг куба

```
A1 = [2, 0, 0, 0;
      0, 1, 0, 0;
      0, 0, 2, 0;
      0, 0, 0, 1];
A2 = [1, 0, 0, 1;
      0, 1, 0, 1;
      0, 0, 1, 1;
      0, 0, 0, 1];
verticesCube2 = A1*A2*verticesCube;
DrawShape(verticesCube, facesCube, 'blue');
DrawShape(verticesCube2, facesCube, 'green');
```

Выполнение можем видеть на рисунке 6:

4 Вращение

А что если мы хотим повернуть наш куб вокруг какой-то из осей? Из курса линейной алгебры мы уже знаем, как задать соответствующую матрицу, осталось лишь подействовать ею на матрицу вершин нашего куба. Хорошо, а что если мы хотим задать повернуть наш

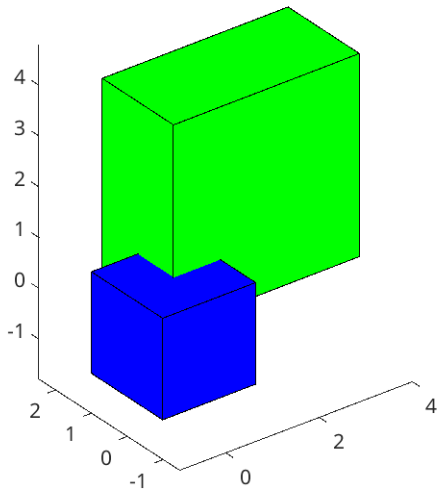


Рис. 6: Сдвиг и масштабирование куба

куб сначала относительно Ox, а потом относительно Oy? Всегда ли эти композиция этих преобразований оказывается равна. В 2D случае это верно в силу того, что у нас существует только одно вращение, а углы складываются (это некий вырожденный случай в мире вращений). Однако в 3D это не так:

```
angleOx = pi/3;
rotOx = [1,            0,             0, 0;
         0, cos(angleOx), -sin(angleOx), 0;
         0, sin(angleOx),  cos(angleOx), 0;
         0,            0,             0, 1];
angleOy = pi/6;
rotOy = [cos(angleOy), 0, sin(angleOy), 0;
                    0, 1,            0, 0;
        -sin(angleOy), 0, cos(angleOy), 0;
                    0, 0,            0, 1];
angleOz = 0;
rotOz = [cos(angleOz), -sin(angleOz), 0, 0;
         sin(angleOz),  cos(angleOz), 0, 0;
                    0,            0, 1, 0;
                    0,            0, 0, 1];
verticesCube = rotOz*rotOx*rotOy*verticesCube;
verticesCube2 = rotOz*rotOy*rotOx*verticesCube;
```

Проверим, будут ли наши кубы совмещены:

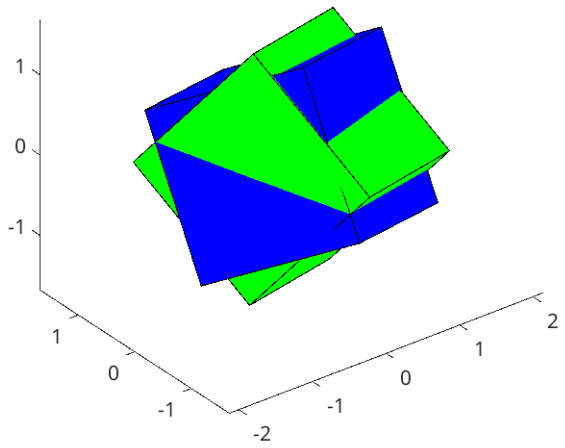


Рис. 7: Некоммутативные вращения

То есть вращения в 3D не коммутативны. Всего же независимых вращений существует, как несложно догадаться, 3, так как независимых осей в 3D всего 3 (на то оно и 3D).

5 Вращение около одной вершины

Задача сводится к последовательному применению следующих преобразований: смещения вершины в центр, соответствующего поворота, а после обратного смещения в исходное положение. Приступим к реализации (будем вращать куб относительно вершины $(1, 1, 1)$):

```
A = [1, 0, 0, -1;
     0, 1, 0, -1;
     0, 0, 1, -1;
     0, 0, 0, 1];
angleOx = pi/3;
rotOx = [1,            0,             0, 0;
         0, cos(angleOx), -sin(angleOx), 0;
```

```
8             0, sin(angleOx),  cos(angleOx), 0;
9             0,            0,            0, 1];
10 angleOy = pi/3;
11 rotOy = [ cos(angleOy), 0, sin(angleOy), 0;
12                        0, 1,            0, 0;
13           -sin(angleOy), 0, cos(angleOy), 0;
14                        0, 0,            0, 1];
15 angleOz = pi/3;
16 rotOz = [cos(angleOz), -sin(angleOz), 0, 0;
17           sin(angleOz),  cos(angleOz), 0, 0;
18                      0,            0, 1, 0;
19                      0,            0, 0, 1];
20 verticesCube2 = A^(-1)*rotOz*rotOx*A*verticesCube;
21
22 DrawShape(verticesCube, facesCube, 'blue');
23 DrawShape(verticesCube2, facesCube, 'green');
```

Результат следующий:

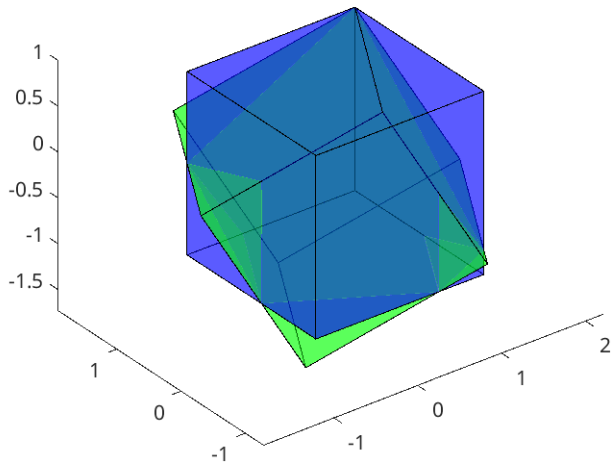


Рис. 8: Некоммутативные вращения

Получили именно то, что мы и хотели! Заметим также, что исходная функция была немного изменена, была добавлена прозрачность 0.4 на отрисовку с помощью добавления дополнительного аргумента FaceAlpha у функции patch.

6 Камера

Вооружившись всеми полученными знаниями, можем приступить к реализации камеры (точнее её симуляции, так как у нас нет непосредственного метода её реализации). Начнём с добавления на сцену нескольких кубиков:

```
verticesCube = [
    -1,  1,  1, -1, -1,  1, 1, -1;
    -1, -1,  1,  1, -1, -1, 1,  1;
    -1, -1, -1, -1,  1,  1, 1,  1;
    1,  1,  1,  1,  1,  1, 1,  1
    ];

facesCube = [
    1, 2, 6, 5;
    2, 3, 7, 6;
    3, 4, 8, 7;
    4, 1, 5, 8;
    1, 2, 3, 4;
    5, 6, 7, 8];

    cube1mat = [1, 0, 0, -5;
    0, 1, 0,  0;
    0, 0, 1,  0;
    0, 0, 0,  1];

    cube2mat = [1, 0, 0, 5;
    0, 1, 0,  1;
    0, 0, 1,  0;
    0, 0, 0,  1];

    cube3mat = [1, 0, 0, 0;
    0, 1, 0,  4;
    0, 0, 1,  5;
    0, 0, 0,  1];

    cube4mat = [1, 0, 0, 0;
    0, 1, 0,  0;
    0, 0, 1,  0;
    0, 0, 0,  1];

    cube1 = cube1mat*verticesCube;
    cube2 = cube2mat*verticesCube;
    cube3 = cube3mat*verticesCube;
    cube4 = cube4mat*verticesCube;
    DrawShape(cube1, facesCube, 'blue');
```

```
41      DrawShape(cube2, facesCube, 'yellow');
42      DrawShape(cube3, facesCube, 'green');
43      DrawShape(cube4, facesCube, 'k');
44
45      axis equal;
46      view([0, 90]);
47      function DrawShape(vertices, faces, color)
48          patch('Vertices', (vertices(1:3,:)./vertices(4,:))', ...
                'Faces', faces, 'FaceColor', color, 'FaceAlpha', 0.4);
49      end
```

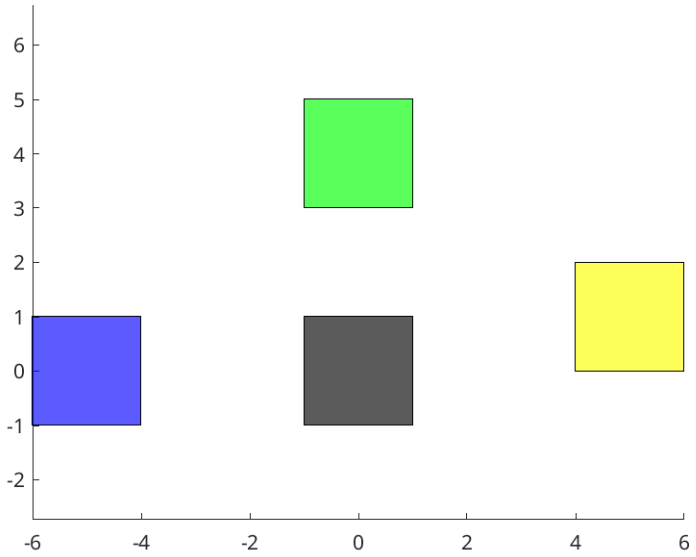


Рис. 9: Размещение кубиков

Уже упоминалось, что у нас будет происходить не непосредственно движение камеры, а лишь её имитация, то есть если мы двигаем камеру в положительном направлении Oy на 5, то нам необходимо сместить все объекты на те же 5 пунктов в противоположном направлении, то есть взять обратную матрицу от движения камеры и применить её к объектам! Именно поэтому мы лишь моделируем движение камеры, тогда как по сути двигается не она, а сами объекты.

Итак, реализуем вышеописанное движение камеры:

```
camera_move = [1, 0, 0, 5;
               0, 1, 0, 0;
               0, 0, 1, 0;
               0, 0, 0, 1];
obj_move = camera_move^(-1);

cube1 = obj_move*cube1mat*verticesCube;
cube2 = obj_move*cube2mat*verticesCube;
cube3 = obj_move*cube3mat*verticesCube;
cube4 = obj_move*cube4mat*verticesCube;
DrawShape(cube1, facesCube, 'blue');
DrawShape(cube2, facesCube, 'yellow');
DrawShape(cube3, facesCube, 'green');
DrawShape(cube4, facesCube, 'k');
```

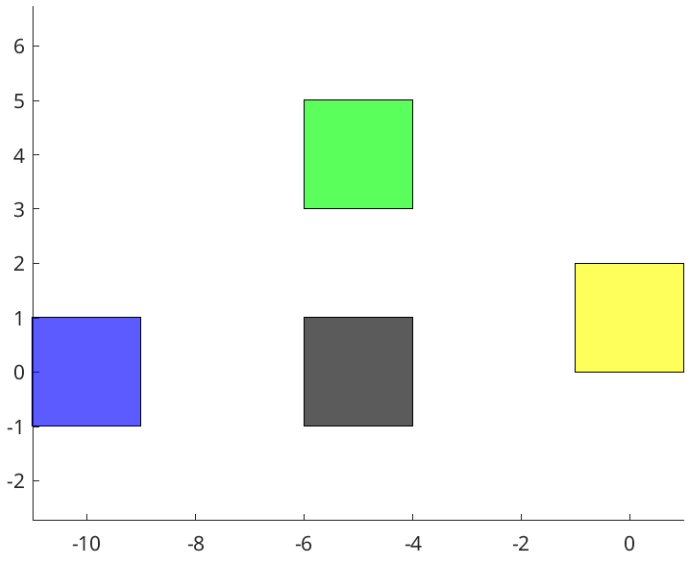


Рис. 10: Движение камеры по Oy

Теперь повернём камеру относительно всё той же Oy, а также добавим сетку (так, мне кажется, лучше видны вращения):

```
camera_move = [1, 0, 0, 5;
               0, 1, 0, 0;
               0, 0, 1, 0;
               0, 0, 0, 1];
angle = pi/8;
camera_rotate = [cos(angle), 0, sin(angle), 0;
```

```
                                0,  1,           0,  0;
                    -sin(angle),  0,  cos(angle),  0;
                                0,  0,           0,  1];
obj_move = camera_move^(-1);
obj_rotate = camera_rotate^(-1);

cube1 = obj_move*obj_rotate*cube1mat*verticesCube;
cube2 = obj_move*obj_rotate*cube2mat*verticesCube;
cube3 = obj_move*obj_rotate*cube3mat*verticesCube;
cube4 = obj_move*obj_rotate*cube4mat*verticesCube;
DrawShape(cube1, facesCube, 'blue');
DrawShape(cube2, facesCube, 'yellow');
DrawShape(cube3, facesCube, 'green');
DrawShape(cube4, facesCube, 'k');
```

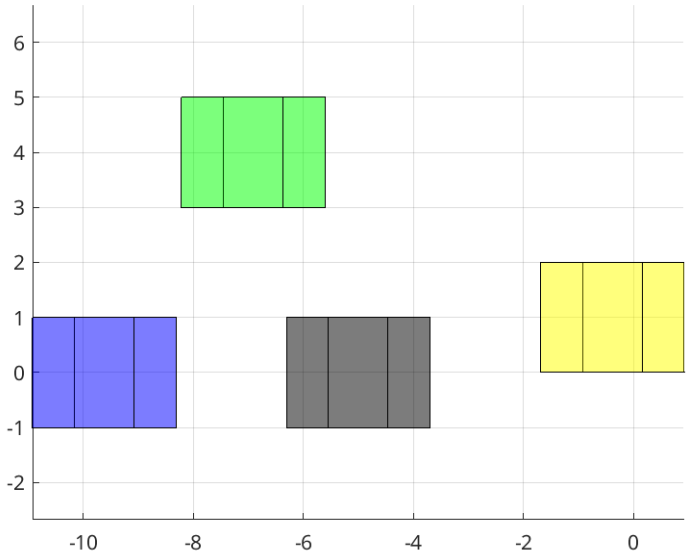


Рис. 11: Движение и вращение камеры относительно Oy

Видим, что движения и вращения происходят успешно, всё работает!

Однако не возникает такого ощущения, что наша камера выглядит как-то топорно и неестественно? С этим нам поможет справиться перспектива.

7 Перспектива

Зададим следующую матрицу:

```
perspective = [1, 0,    0, 0;
               0, 1,    0, 0;
               0, 0,    1, 0;
               0, 0, 0.04, 1];
```

Число в последней строке задаёт перспективное искажение при проецировании на Oxy (отрицательно число в столбце Oz дало бы искажение для поворота камеры по Ox, мы же вращаем Oy, а значит, ставим +). Чем больше это число - тем более искаженной получается картинка. Выбираем, куда ставить число, методом исключения осей, участвующих в проекции (то есть если будем проекцировать на Oxz, то число поставится в стобце Oy).

Наши же кубы под действием этой матрицы в трёхмерном пространстве больше не будут являться таковыми. Казалось бы, одни минусы, но полученная матрица позволяет сделать нашу картинку более естественной, исказив пространство. Посмотрим на результат:

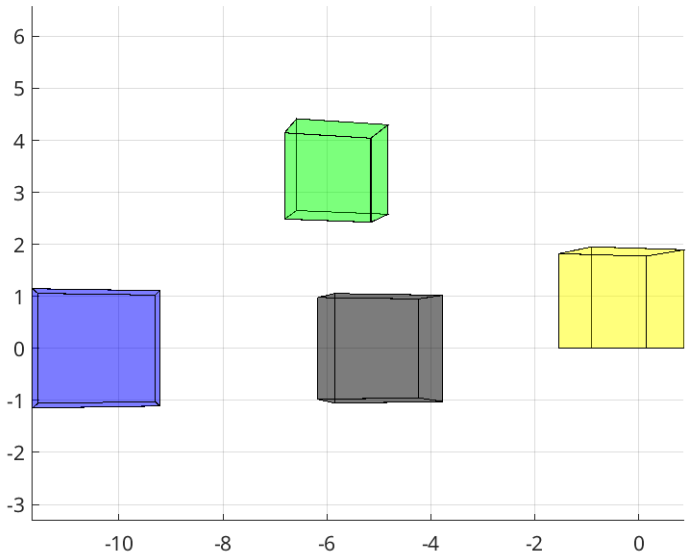


Рис. 12: Естественное движение и вращение камеры относительно Oy

Мы исправили проблему с неестественностью движения камеры! Теперь можем смело вращать и двигать нашу камеру, не боясь показать результаты нашей работы.

Посмотрим теперь, что случилось с нашими кубами в пространстве:

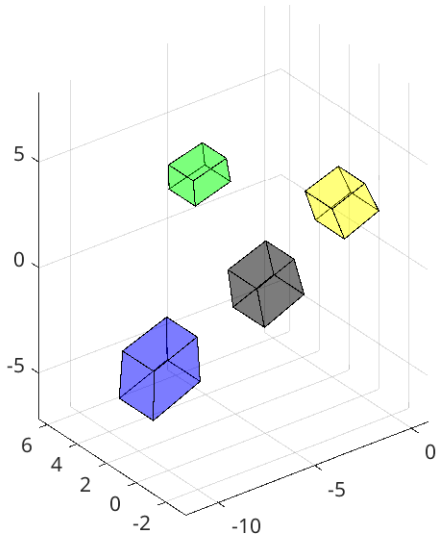


Рис. 13: Искажение кубов в пространстве

Как и говорилось выше, фигуры исказились, но при этом мы выиграли при проецировании в естественности движения камеры.

На этом всё. Спасибо!